Object Oriented Programming

Course Code: 25CSH-208

UNIT-2 CHAPTER -2.1

IO PACAKAGE

UNIT-2	**IO PACAKGE AND MULTITHREADING**	Contact Hours:15
IO package	Introduction to IO Package and Streams, Input streams, Output streams, Object serialization, De-serialization, Filter and pipe streams, IOFiles, Sample programs on IO files.	
Multi-threading	Thread Lifecycle, Multi-threading, Advantages and Issues of Multi-threading, Simple thread program, Thread-synchronization.	

INTRODUCTION TO IO PACKAGES & STREAMS

- Java's input/output system is based on the concept of streams.
- A stream is a path along which data flows from a source to a destination.
- Source might be a keyboard, file, or program, etc.
- Destination might be the console(screen), file, program, etc.
- Reason for using a stream as the basis for input or output operations is to make the program code for these operations independent of the device involved.
- The stream-based IO operations are faster than normal IO operations.
- The Stream is defined in the **java.io package**.
- The Java streams support many different kinds of data, including simple bytes, primitive data types, localized characters, and objects.
- The advantages of using streams:

• One doesn't need to worry about the detailed mechanisms of each device, as it is automatically taken care of behind the scenes.

- Your program works for a variety of Input/Output devices without any changes to the code.

➢ In Java, the stream-based IO operations are performed using two separate streams: **input stream** and **output stream**.

➢ The **input stream** is used for input operations, and the **output stream** is used for output operations.

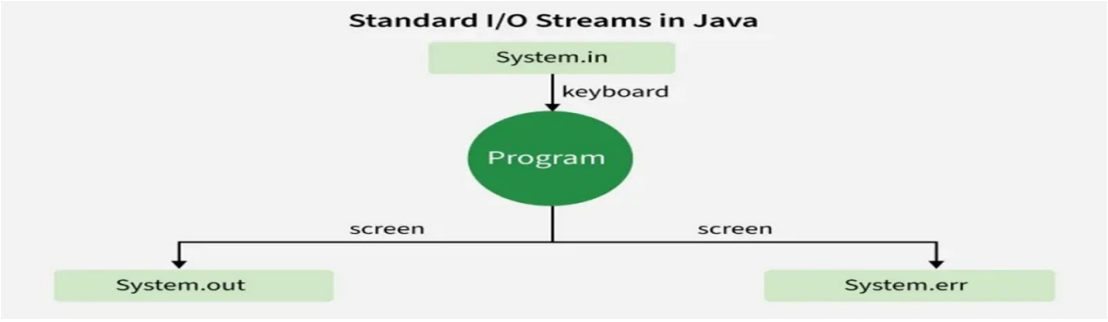
➢ The Java stream is composed of bytes.

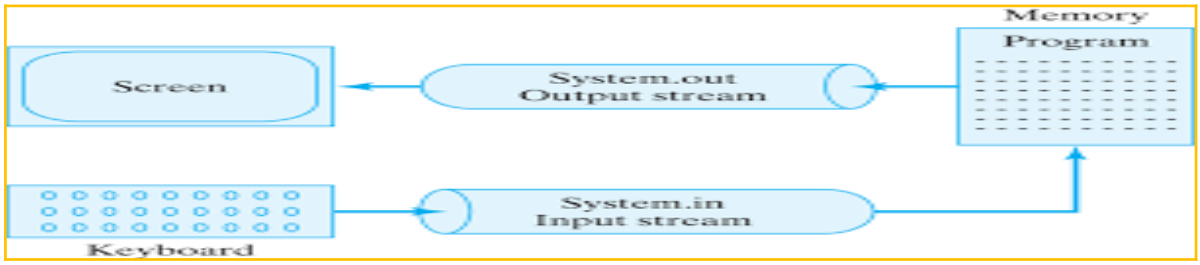


➢ System class of the **java.lang package** contains three predefined streams.

➢ In Java, every program creates 3 streams automatically, and these streams are attached to the console.

- **System.out**: standard output stream for console output operations.
- **System.in**: standard input stream for console input operations.
- **System.err**: standard error stream for console error output operations.





Some Functions used with Predefined Streams

1. System.in

- System.in is an InputStream, so it provides methods from the InputStream class. Commonly used ones are:
- **int read():** reads one byte of data.
- **int read(byte[] b):** reads bytes into an array.

2. System.out

- **print():** This Java method displays text on the console, keeping the cursor at the end of the printed text so the next output continues from the same line.
- **Syntax:** *System.out.print(parameter);*
- **println():** This method in Java is also used to display text on the console and the cursor moves to the start of the next line at the console.

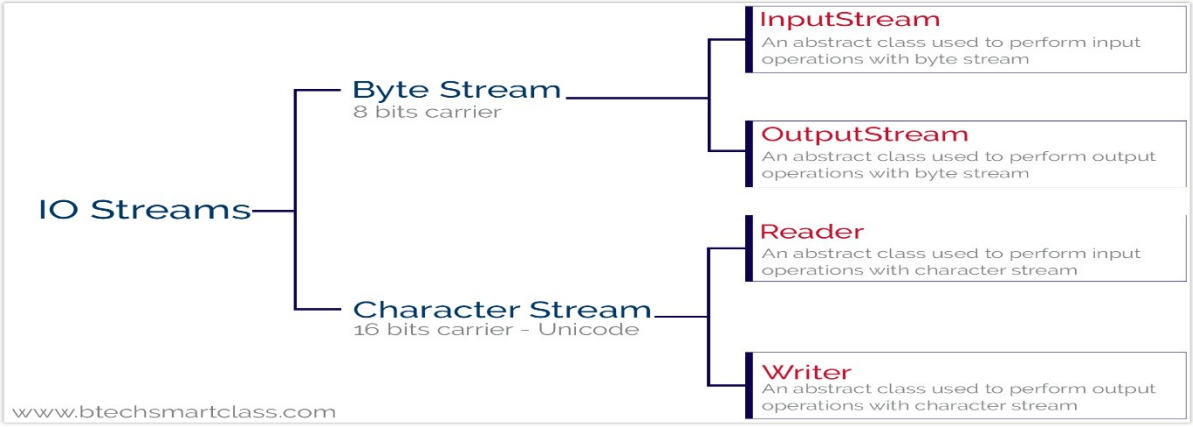
3. System.err

- It is used to display the error messages. It works similarly to System.out with print(), println(), and printf() methods.

Java provides two types of streams, they are as follows.

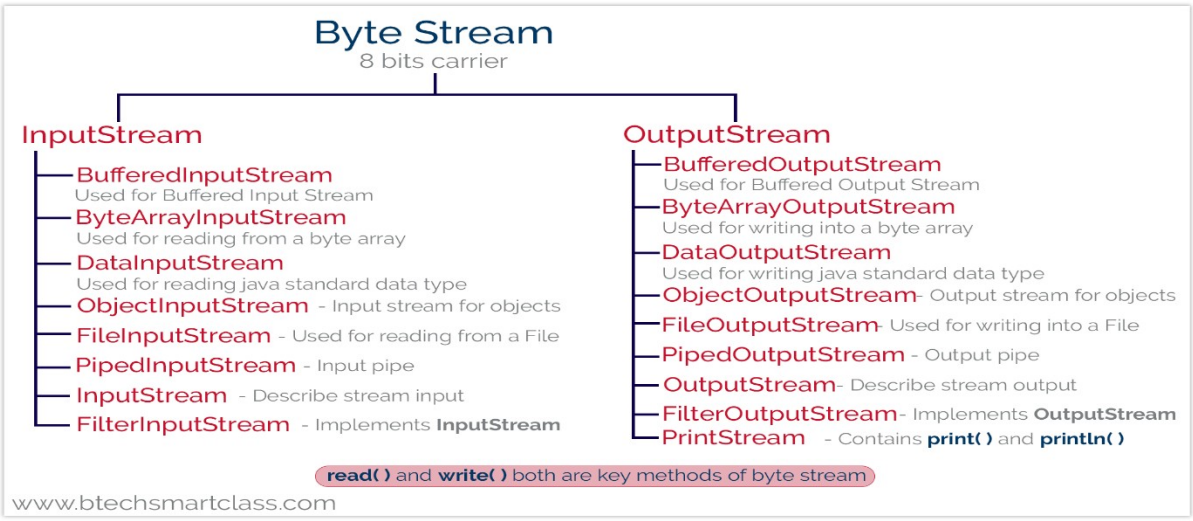
- Byte Stream
- Character Stream

The following picture shows how streams are categorized, and various built-in classes used by the java IO system.



1.1 Byte Streams in java

- In Java, the streams that are used to handling binary data are known as **byte streams**
- When you write to a binary stream, the data is written to the stream as a series of bytes exactly as it appears in memory. No transformation of data takes place.
- The value of the same primitive type is represented by the same number of bytes in a binary file. For example, all **int** values occupy four bytes each in a binary file.
- In the Java 1.0 version, all IO operations were byte oriented; there was no other stream (character stream).
- The Java **byte stream** is defined by two abstract superclasses, **Input Stream** and **Output Stream**.
- To use an abstract class, you must subclass it and instantiate the subclass.
- The Input Stream and Output Stream classes have several concrete classes to perform various IO operations based on the byte stream.
- The **Input Stream** class is used for byte stream based input operations, and the Output Stream Class is used for byte stream based output operations.



1.1.1 InputStream Classes

- The java.io.InputStream class is the abstract superclass of all byte input stream classes.

- It declares methods to read one or more bytes of data, checking how many bytes of data are available to be read, marking the position in a stream, resetting back to that position, closing the stream, skipping over input, etc.
- Some of the methods of the InputStream class are shown in the given table.
- **Subclasses of InputStream**

  •FileInputStream

  •ByteArrayInputStream

  •ObjectInputStream

  •SequenceInputStream

  •PipedInputStream

  •FilterInputStream

  •StringBufferInputStream

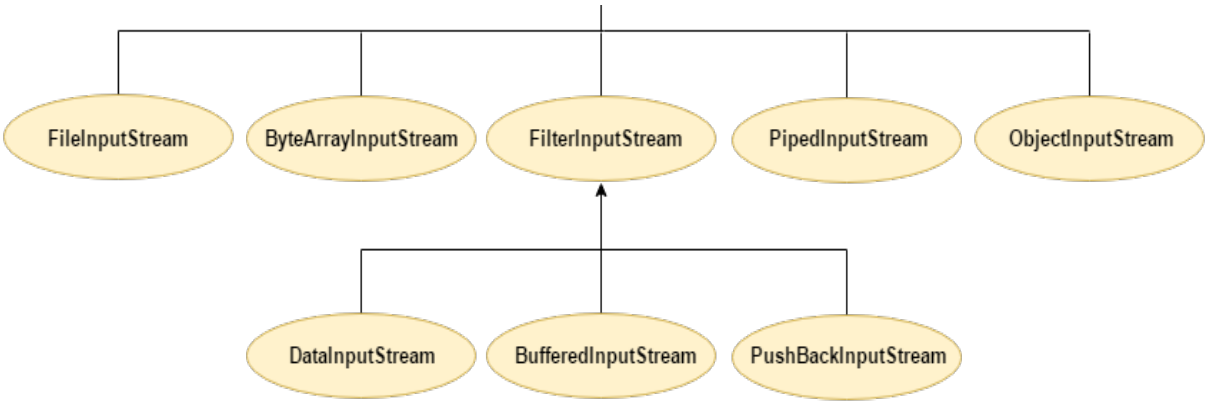
- **Create an InputStream**

InputStream object1 = new FileInputStream();

Here, we have created an input stream using FileInputStream. It is because InputStream is an abstract class. Hence, we cannot create an object of InputStream.

void mark(int *numBytes*)	Places a mark at the current point in the input stream that will remain valid until *numBytes* bytes are read.
boolean markSupported( )	Returns **true** if **mark( )/reset( )** are supported by the invoking stream.
int read( )	Returns an integer representation of the next available byte of input. −1 is returned when the end of the stream is encountered.
int read(byte *buffer*[ ])	Attempts to read up to *buffer.length* bytes into *buffer* and returns the actual number of bytes that were successfully read. −1 is returned when the end of the stream is encountered.
int read(byte *buffer*[ ], int *offset*, int *numBytes*)	Attempts to read up to *numBytes* bytes into *buffer* starting at *buffer*[*offset*], returning the number of bytes successfully read. −1 is returned when the end of the stream is encountered.
void reset( )	Resets the input pointer to the previously set mark.
long skip(long *numBytes*)	Ignores (that is, skips) *numBytes* bytes of input, returning the number of bytes actually ignored.

InputStream class Hierarchy



- The InputStream class has several direct subclasses as shown in the figure.
- These subclasses override the InputStream methods and provide additional functionality.
- The FilterInputStream subclass has further direct subclasses that provide a more specialized way of filtering or transforming data from the input stream

1.1.2 OutputStream Class

- The java.io.OutputStream class is the abstract superclass representing an output stream of bytes.
- It provides the fundamental methods needed to write data to various output destinations.
- All OutputStream classes inherit from this base class.
- OutputStream class is an abstract class that cannot be instantiated directly.
- It provides the template for all byte output operations.
- It includes key methods writing bytes, flushing buffers, and closing the streams.
- All concrete subclasses implements these methods.
- **Subclasses of OutputStream**

- FileOutputStream
- ByteArrayOutputStream
- ObjectOutputStream
- PipedOutputStream
- FilterOutputStream
- StringBufferedOutputStream
- SequenceOutputStream

➢ Create an OutputStream

OutputStream object = new FileOutputStream();

Here, we have created an object of an output stream using FileOutputStream. It is because OutputStream is an abstract class, so we cannot create an object of OutputStream.

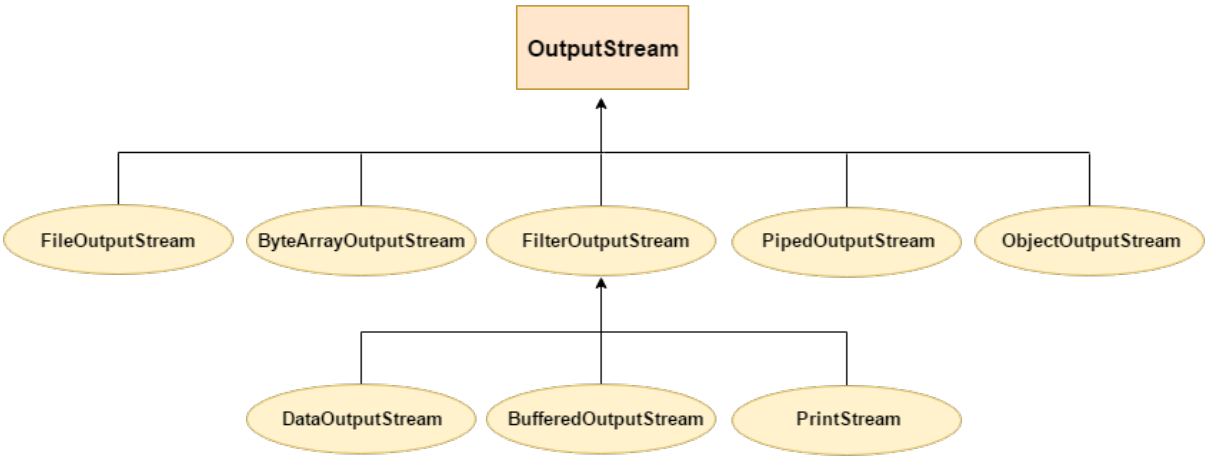
➢ **Key Methods**

The OutputStream class provides several key methods that are essential for writing data:

- **write(int b)**: Writes the specified byte to this output stream. The general contract for write is that one byte is written to the output stream. The byte to be written is the eight low-order bits of the argument b. The 24 high-order bits of b are ignored.
- **write(byte[] b)**: Writes b.length bytes from the specified byte array to this output stream.
- **write(byte[] b, int off, int len)**: Writes len bytes from the specified byte array starting at offset off to this output stream.
- **flush()**: Flushes this output stream and forces any buffered output bytes to be written out.
- **close()**: Closes this output stream and releases any system resources associated with this stream.

Method	Purpose
flush()	Flushes this output stream and forces any buffered output bytes to be written out.
write(byte[] b)	Writes `b.length` bytes from the specified byte array to this output stream.
write(byte[] b, int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this output stream.
write(int b)	Writes the specified byte to this output stream.
close()	Closes this output stream and releases any system resources associated with this stream.

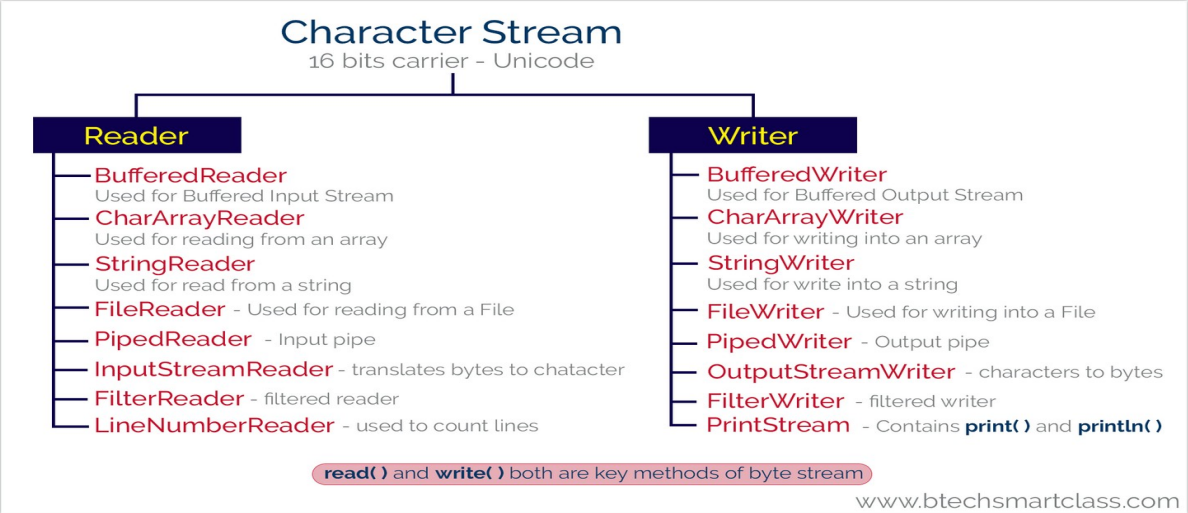
OutputStream Classes Hierarchy



- The OutputStream class has several direct subclasses, as shown in the figure.
- These subclasses override the OutputStream methods and provide additional functionality
- The FilterOutputStream subclass has further direct subclasses that provide a more specialized way of filtering or transforming data from the output stream

1.2 CharacterStream Classes in Java

- The CharacterStream classes are used for performing I/O on Characters rather than on bytes.
- CharacterStream classes are defined by using two class hierarchies.
- At the top of these are two classes: Reader & Writer
- The Reader Class is an abstract superclass of all character input stream Classes.
- The reader class converts bytes from a byte stream into characters in Java's Unicode representation.
- The Writer class is an abstract superclass of all character output stream Classes.
- The writer class converts characters into bytes and then write those bytes to a byte output stream.



1.2.1 Reader Class

- The Reader class in Java is an abstract class that represents a stream of characters.
- It is part of the java.io package and serves as the superclass for various subclasses that provide specific functionalities for reading character streams.
- Since Reader is an abstract class, it cannot be instantiated directly. Instead, its subclasses are used to read data from different sources.
- **Subclasses of Reader**

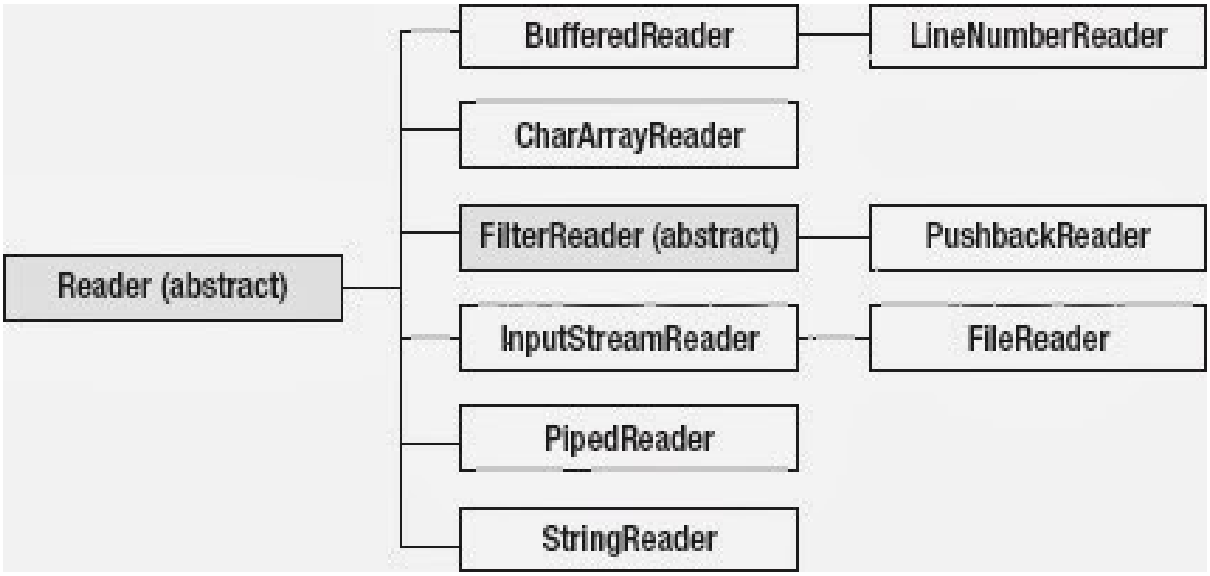
- **BufferedReader**: Provides buffering for efficient reading of characters, arrays, and lines.
- **InputStreamReader**: Converts byte streams to character streams using a specified charset.
- **FileReader**: Reads characters from files.
- **StringReader**: Reads characters from a string.

➢ **Methods of Reader Class**

The Reader class provides several methods that are implemented by its subclasses. Some commonly used methods include:

- **read()**: Reads a single character from the stream.
- **read(char[] array)**: Reads characters from the stream and stores them in the specified array.

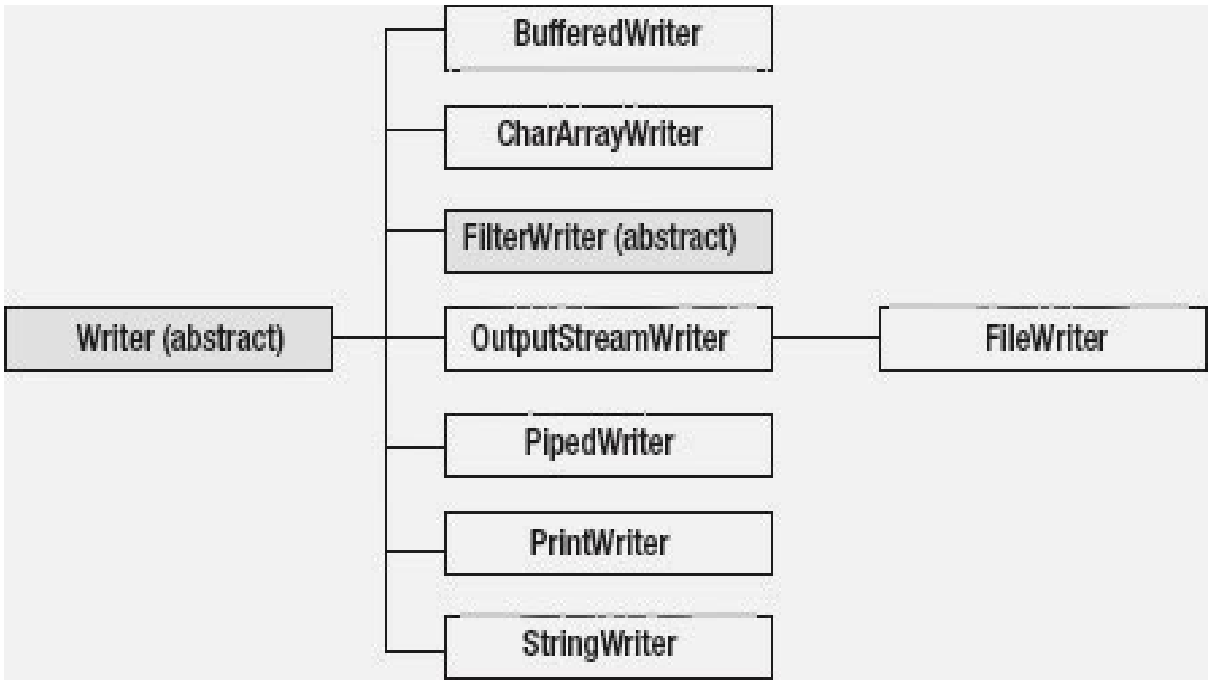
- **read(char[] array, int start, int length)**: Reads a specified number of characters from the stream and stores them in the array starting from the specified index.
- **ready()**: Checks if the reader is ready to be read.
- **mark()**: Marks the current position in the stream.
- **reset()**: Resets the stream to the previously marked position.
- **skip()**: Skips a specified number of characters in the stream.



1.2.2 Writer Class

- The **Writer** class in Java, part of the java.io package, is an **abstract class** designed for writing character streams.
- It implements **Appendable**, **Closeable**, and **Flushable**, making it suitable for writing characters, arrays, and strings to files, memory, or other output destinations.
- Since it's abstract, you cannot instantiate it directly; instead, you use subclasses like **FileWriter**, **BufferedWriter**, **PrintWriter**, or **StringWriter**.
- **Key Features:**

- **Abstract methods**: write(char[] cbuf, int off, int len), flush(), close()
- **Concrete methods**: Overloaded write() for single characters, strings, and arrays; append() methods for adding characters or sequences.
- **Thread-safety**: Synchronization can be done using the internal lock object.


BufferedWriter
CharArrayWriter
FilterWriter (abstract)
Writer (abstract)
OutputStreamWriter
FileWriter
PipedWriter
PrintWriter
StringWriter